

Logbook

RTOS Q4
(Linux Device Driver)

Contents

1. Before we start	3
2. Trying out the sample	4
2.1. The sample driver from Apriorit	4
2.1.1. pr_notice fun fact :	5
2.2. The Conclutions with Apriorit	9
3. Making my own device driver.	9

1. Before we start

This is the second logbook for RealTime Operating System. This assignment goals is to make us understand more about making a Linux Driver.

And this is my logbook of trying to make things work

2. Trying out the sample

2.1. The sample driver from Apriorit

from the assignment, we have to try out the sample linux driver from Apriorit. And this is what i get from the first simple driver explanation in the web (also in GitHub).

```
#include "device_file.h"
#include <linux/init.h>      /* module_init, module_exit */
#include <linux/module.h>    /* version info, MODULE_LICENSE, MODULE_AUTHOR */

MODULE_DESCRIPTION("Simple Linux driver");
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Apriorit, Inc");

static int simple_driver_init(void)
{
    int result = 0;
    pr_notice("Simple-driver: Initialization started\n");

    result = register_device();
    if (result)
    {
        pr_notice("Simple-driver: Failed to register character device\n");
    }
    return result;
}

static void simple_driver_exit(void)
{
    pr_notice("Simple-driver: Exiting\n");
    unregister_device();
}

module_init(simple_driver_init);
module_exit(simple_driver_exit);
```

Let me explain a bit of what is happening here :

- **module_init** → This is the function that tell the driver module loader (insmod, insert module) the function to run. In this case it would run the simple_driver_init function
- **module_exit** → This is the function that tell the driver module remover (rmmod, remove module) the function to run. In this case it would run the simple_driver_exit function
- **MODULES (anything with this in front)** → This is the module descriptor, so if you do modinfo you can see the description listed there.
- **pr_notice** → This is basically like the printf of your normal c file, but this print the notice in your dmesg (fun fact about pr_notice after this)

2.1.1. pr_notice fun fact :

```
printk(KERN_NOTICE "Simple-driver: Initialization started\n");  
pr_notice("Simple-driver: Initialization started\n");
```

this two are the same thing, so pr_notice is just printk with KERN_NOTICE. For me personally, i would just use printk because of muscle memory to type printf

But then after some searching, there is actually more to put in the printk function

- KERN_EMERG ⇒ System is unusable.
- KERN_ERR ⇒ Error conditions (usually used if register_device() fails).
- KERN_WARNING ⇒ Warning flags.
- KERN_INFO ⇒ Informational messages tracking healthy runtime behaviors.
- KERN_NOTICE ⇒ Normal but significant operational alerts.

The code is too long for me to put it here, so you can just open the code from https://github.com/apriorit/SimpleLinuxDriver/blob/master/device_file.c

Let me break it down for you :

```
int register_device(void) {  
    int result = 0;  
  
    pr_notice("Simple-driver: register_device() is called.\n");  
  
    unsigned baseminor = 0;  
    unsigned minor_count_required = 1;  
    result = alloc_chrdev_region(&g_devno, baseminor, minor_count_required, device_name);  
    if (result)  
    {  
        pr_err("Simple-driver: alloc_chrdev_region failed: %d\n", result);  
        goto err_out;  
    }  
  
    cdev_init(&g_cdev, &simple_driver_fops);  
    g_cdev.owner = THIS_MODULE;  
    result = cdev_add(&g_cdev, g_devno, minor_count_required);  
    if (result)  
    {  
        pr_err("Simple-driver: cdev_add failed: %d\n", result);  
        goto err_unregister_chrdev_region;  
    }  
  
    g_class = class_create(class_name);  
    if (IS_ERR(g_class))  
    {  
        result = PTR_ERR(g_class);  
        pr_err("Simple-driver: class_create failed: %d\n", result);  
        goto err_cdev_del;  
    }  
  
    g_device = device_create(g_class, NULL, g_devno, NULL, device_name);  
    if (IS_ERR(g_device))  
    {  
        result = PTR_ERR(g_device);  
        pr_err("Simple-driver: device_create failed: %d\n", result);  
        goto err_class_destroy;  
    }  
}
```

```
pr_notice("Simple-driver: Registered character device with major number = %i, minor number = %i\n", MAJOR(g_devno), MINOR(g_devno));
return 0;

err_class_destroy:
if (!IS_ERR_OR_NULL(g_class))
{
    class_destroy(g_class);
    g_class = NULL;
}

err_cdev_del:
cdev_del(&g_cdev);

err_unregister_chrdev_region:
if (g_devno)
{
    unsigned minor_count_allocated = 1;
    unregister_chrdev_region(g_devno, minor_count_allocated);
    g_devno = 0;
}

err_out:
return result;
}
```

This function is set for the module_init calling. In this function there is :

- 1. Allocate the identification (alloc_chrdev_region)
- 2. Initialize the internal kernel structure (cdev_add)
- 3. Create the system class functions (class_create)
- 4. Mount the user interface device file for I/O (device_create)

But then why is the code so long ? **Because of the fallback that we have**

Imagine if the code is crashing in step 4, you cannot just stop there, you have to clear all the steps before. Therefore you use a goto functions and make err_ functions to handle the clear up for you. So this initializer function really make sure that you didn't leak any memory or anything bad that we don't want.

```
void unregister_device(void)
{
    pr_notice("Simple-driver: unregister_device() is called\n");

    if (!IS_ERR_OR_NULL(g_device))
    {
        device_destroy(g_class, g_devno);
        g_device = NULL;
    }

    if (!IS_ERR_OR_NULL(g_class))
    {
        class_destroy(g_class);
        g_class = NULL;
    }

    cdev_del(&g_cdev);
}
```

```
if (g_devno)
{
    unsigned minor_count_allocated = 1;
    unregister_chrdev_region(g_devno, minor_count_allocated);
    g_devno = 0;
}
pr_info("Simple-driver: Unregistered\n");
}
```

This is the function that was called to remove the kernel module from the loaded kernel module (in other word `rmmmod`). Here there is not much thing to explain. Just the same way like when you do the error handling in the `register_device` on top.

And the rest of the code

```
static const char g_s_Hello_World_string[] = "Hello world from kernel mode!\n";
static const ssize_t g_s_Hello_World_size = sizeof(g_s_Hello_World_string);

static ssize_t device_file_read(
    struct file *file_ptr, char __user *user_buffer, size_t count, loff_t *position)
{
    pr_notice("Simple-driver: Read from device file offset = %i, read bytes count = %u\n",
        (int)*position, (unsigned int)count);

    if (*position >= g_s_Hello_World_size)
        return 0;

    if (*position + count > g_s_Hello_World_size)
        count = g_s_Hello_World_size - *position;

    if (copy_to_user(user_buffer, g_s_Hello_World_string + *position, count) != 0)
        return -EFAULT;

    *position += count;
    return count;
}
```

When analyzing the runtime behavior using the `dmesg` tool, I noticed the driver outputs two lines showing the offset and read bytes count. I broke down what these numbers actually mean during the file access loop:

- **Read Bytes Count:** This value represents the maximum size of the memory buffer that the application (in this case, the `cat` command) has allocated to hold incoming data. The system defaults this to 4096 bytes because that is the standard size of a single memory page in Linux. The utility requests this full amount even though our message is much smaller, simply because it does not know the file's size in advance.
- **Offset:** This serves as a positional tracking cursor or bookmark. It tells the driver exactly which byte index to start reading from inside the kernel string array. An offset of 0 means reading starts at the first character.
- **The Dual-Read Loop Mechanism:** The log records two distinct access attempts because of how the reading loop operates in the background:

1. **First Pass (offset = 0):** The tool requests data from the beginning. The driver copies the 30-byte message over and shifts the tracking offset position directly to index 30.
2. **Second Pass (offset = 30):** The tool loops back to verify if more text exists. It requests more data starting at position 30. The driver checks the boundaries, sees the offset has reached the end of the text, and returns a value of 0. This zero return acts as the standard End-of-File signal, telling the reading tool to stop and exit safely.

But, how to tell the operating system which code to execute when we run the kernel ?

To explain how the operating system knows which code to execute when we run commands, we have to look at the `simple_driver_fops` structure.

```
static const struct file_operations simple_driver_fops =
{
    .owner = THIS_MODULE,
    .read = device_file_read,
};
```

```
static const char device_name[] = "simple-driver";
static const char class_name[] = "simple-driver-class";
dev_t g_devno = 0;
struct cdev g_cdev = {};
static struct class *g_class = NULL;
static struct device *g_device = NULL;
```

To make this mechanism easy to visualize, think of this structure as a **Standardized Services Menu** at a bank counter:

- A customer walking up to the bank window represents a User Space program (like the cat command).
- The customer cannot just walk into the secure vault or tell the bank tellers to run custom errands. They have to choose a standard operation directly from the laminated service menu sitting on the counter.

I broke down how this structure maps those menu choices to our actual backend driver instructions:

- **The VFS Bridge (Treating Devices like Files):** In Linux, there is a core design rule: “Everything is a file.” When a terminal command tries to talk to our device path at `/dev/simple-driver`, it passes through a system boundary called the Virtual File System (VFS). The VFS picks up the request and instantly checks our `file_operations` structure to see what specific rules and actions this custom device file is allowed to support.
- **Module Safety Allocation (.owner = THIS_MODULE):** This line acts as a building safety lock. It tells the operating system exactly which module owns this operational menu
- **Function Redirection (.read = device_file_read):** This is the literal line that glues the standard operating system read action to our custom C function. When the user selects the “Read” option from the system menu, the VFS looks at this pointer and redirects the execution straight down into our `device_file_read` function. Because we did not define any `.write` or `.open` pathways on this menu, the Linux kernel automatically handles those missing choices with built-in safety rejections. This

completely blocks any write requests by default and turns our code into a secure, read-only device.

2.2. The Conclutions with Apriorit

Basically, when we compile it we will get like a .ko file and that is the one that you can do insmod. The github that we can pull from apriorit actually have this in the cmake

```
TARGET_MODULE:=simple-module

# If we running by kernel building system
ifneq ($(KERNELRELEASE),)
    $(TARGET_MODULE)-objs := main.o device_file.o
    obj-m := $(TARGET_MODULE).o

# If we are running without kernel build system
else
    BUILDSYSTEM_DIR:=/lib/modules/$(shell uname -r)/build
    PWD:=$(shell pwd)

all :
# run kernel build system to make module
$(MAKE) -C $(BUILDSYSTEM_DIR) M=$(PWD) modules

clean:
# run kernel build system to cleanup in current directory
$(MAKE) -C $(BUILDSYSTEM_DIR) M=$(PWD) clean

load:
    insmod ./${TARGET_MODULE}.ko

unload:
    rmmod ./${TARGET_MODULE}.ko

endif
```

See the load and unload part, its using insmod and rmmod.

And from that, after we are done compiling it, it will make a .ko file which we can use for loading it to the active kernel module it will load the main.c (the first code that i gave on top), and it will utilize the module init in the device_file.c

3. Making my own device driver.

In this part of the logbook, i will continue with the assignment 1b which is to change the name to my own name bastian-driver